

MapleStory DX11 2D Engine

언리얼 아키텍처 재현 프로젝트

STL·예외·RTTI 없이 C++17로 직접 구현한 2D 게임 엔진. 언리얼 엔진의 실제 내부 자료구조와 설계 원리를 원리 단위로 재현하고, Windows/Linux 교차 검증과 실전 디버깅 사례로 완성도를 증명한다.

TARGET GAME

MapleStory 스타일 2D 플랫폼어 RPG

LANGUAGE / STANDARD

C++17 (No STL / No Exceptions / No RTTI)

CURRENT STATUS

LAYER 1 코어 시스템 완료 (**Phase 0-7.7** + 최적화 5종)

NEXT MILESTONE

Phase 8 — DirectX11 Renderer

TArray / TSet / TMap

TSparseArray

FName Interning

Bin Allocator

Atomic RefCounting

UClass / Cast<T>

Gameplay Ability System

1 핵심 성과 요약

86

UNIT TESTS

5

CORE SYSTEMS
OPTIMIZED

3

DEBUGGED REAL-
WORLD BUGS

3+3

BUILD CONFIGS (WIN /
LINUX)

언리얼 실제 내부 구조 재현

TSparseArray 기반 TSet/TMap 해시 체인,
FName 인터닝 풀, FRefCountBlock 컨트롤 블록
까지 — 문서가 아닌 실제 UE5 소스 구조와 원리 단
위로 대조 검증 완료.

이중 플랫폼 교차 검증

Windows Debug/Release 필수 통과 + Linux g+
+ 3구성 (O0/O2/O2+NDEBUG) + valgrind
memcheck. 컴파일러·힙 구현이 다른 두 환경에서
서로 다른 버그를 잡아낸 실사례 보유.

컴파일러 레벨 실전 디버깅

MSVC Debug 인라인 코드생성 버그를 콜스택 부재
와 크래시 불변성만으로 추론·확정. noinline 격리로
해결.

구성 의존적 메모리 버그 해결

순환 참조 Release 전용 힙 손상을
std::shared_ptr와 동일한 암묵적 weak 참조 패턴
으로 해결. 원인을 주소 계산으로 수치 확정.

2 구현 시스템 개요

시스템	핵심 설계
Memory	전역 GMalloc + IAllocator 교체형 구조 · FMallocBinned(16~512B Bin, 64KB 페이지 정렬 + 포인터 마스킹 역추적) · sized delete 대응
Containers	TArray(+TInlineAllocator<N>, EBO) · TSparseArray(안정 인덱스+프리 리스트) · TSet/TMap(해시 버킷 인덱스 체인, 톰스톤 없음)
String	FString(동적 wchar 버퍼) · FName+FNamePool(djb2 콘텐츠 해시 인터닝, 인라인 엔트리 저장, O(1) 비교)
SmartPointer	FRefCountBlock(Shared/Weak 카운트 이원화, 암묵적 weak 참조) · 원자적 증감(MSVC/GCC 분기) · TSharedPtr/TWeakPtr/TSharedRef
Object Model	UClass 상속 체인 + 함수-지역 정적 StaticClass() · Cast<T>/CastChecked/ExactCast (RTTI 없이 IsChildOf) · AActor 컴포넌트 시스템
Timer	FTimerManager — SetTimer(루프/1회)/Clear/Pause/Resume/NextFrame
Logging / Assert	UE_LOG 카테고리 시스템 · check(Debug 전용)/verify(항상 평가)/ensure(1회 발화) 3단 정책
Gameplay Ability System	FGameplayTag 계층 태그 · Instant/Duration/Infinite+Period 이펙트 · 태그 기반 쿨다운/차단 · Add→Multiply→Override 3패스 속성 재계산

3

TSet / TMap — 해시 버킷 인덱스 체인

원소는 TSparseArray에 저장되어 삭제 후에도 인덱스가 안정적이고, pow2 버킷 배열은 해시별 첫 원소 인덱스만 보관한다. 충돌은 원소 내 `m_HashNext` 인덱스로 체인 연결. 삭제는 체인 unlink + 슬롯 반납으로 **O(1)**, 톰스톤 없음 — Open Addressing 방식에서 발생하던 삭제 표식 누적·불필요한 Rehash 문제를 원천 해결했다.

```

buckets[hash & (N-1)] → [3] —m_HashNext→ [0] → INDEX_NONE
                        ↳ 같은 버킷의 원소들이 인덱스 체인으로 연결
Rehash: 원소는 그대로, 버킷 배열만 새로 만들어 인덱스 재링크 (원소 이동 0회)

```

FMallocBinned — 포인터 마스킹 기반 O(1) 할당자

16~512B 크기 클래스 Bin을 64KB 정렬 페이지에서 잘라 쓴다. 해제 시 `ptr & ~(PAGE_SIZE-1)` 마스크 한 번으로 소속 페이지 헤더를 역추적해 Bin을 알아내므로, 블록별 헤더 오버헤드가 0바이트다.

Free(ptr): ptr & ~(65536-1) → 페이지 헤더 → Bin 인덱스 → 프리 리스트 반납

TSharedPtr — 암묵적 Weak 참조로 순환 참조 이중 해제 방지

컨트롤 블록의 `WeakCount` 를 1로 초기화해 "공유 소유자 그룹 전체"의 암묵적 weak 참조를 표현한다. 마지막 Shared 해제 시 Deleter가 실행되는 도중 멤버 TWeakPtr들이 자기 몫을 반납해도 그룹 몫 1이 블록을 지켜주며, **Deleter** 완료 후에야 그룹 몫까지 반납되어 블록이 해제된다 — `std::shared_ptr`와 동일한 패턴으로, 실제 겪은 Release 전용 이중 해제 버그를 해결했다.

4 언리얼 실제 구조 대조 검증

전 서브시스템을 UE5 실제 소스 구조와 원리 단위로 대조 검증했다.

서브시스템	판정	핵심 근거
TArray + TInlineAllocator	동일	할당자 정책 파라미터 주입, 인라인→힙 스펠 — ContainerAllocationPolicies와 동일 원리
TSparseArray	동일	프리 리스트 인덱스 재사용 = UE FreeListLink 구조
TSet / TMap	동일	원소 내 체인(HashNextId) + 버킷 배열 = UE Set.h 알고리즘
FName / FNamePool	동일	인라인 엔트리 + 풀 + 인덱스 비교 = UE NameTypes 핵심
FRefCountBlock	동일	UE FReferenceControllerBase(ThreadSafe)와 동일 설계
DECLARE_CLASS / Cast<T>	동일	GENERATED_BODY 최소 골격 + IsChildOf 캐스트 체인
FMallocBinned	축소	Bin 풀링-페이지 마스킹 일치, UE는 스레드 캐시 등 추가
Gameplay Ability System	축소	개념 계층 전부 대응, 네트워크 예측 계층 미구현(싱글플레이 목표)
TMultiMap	차이	UE는 셋에 중복 키 저장, 본 프로젝트는 TMap<K,TArray<V>> 래퍼

5 트러블슈팅 하이라이트

① TSharedPtr 순환 참조 — Release 전용 힙 손상 (0xC0000374)

증상	Debug는 통과하나 Release에서 순환 참조 테스트가 힙 손상으로 크래시
원인	파괴 중 Deleter 체인 안에서 TWeakPtr가 컨트롤 블록을 먼저 해제 → 이중 해제. Debug 힙의 0xDD 채움이 우연히 증상을 가려 Release에서만 노출
해결	WeakCount를 1로 초기화하는 암묵적 weak 참조 패턴 도입(std::shared_ptr와 동일)

② MSVC Debug 인라인 코드생성 버그 (FName::ToString)

증상	값 반환 시 접근 위반. 함수 내부를 3회 재구성해도 크래시 위치·값이 완전히 동일
원인	콜스택에 해당 함수 프레임이 한 번도 없음 → /Od에서도 초소형 함수가 인라인되어 소스 수정이 생성 코드에 반영 안 됨
해결	함수를 .cpp로 분리 + noline 지정 → 진짜 함수 호출 경계(x64 ABI) 강제로 즉시 해결

③ Release NDEBUG에서 check() 부수효과 증발 (0xC0000005)

증상	Release에서만 컨테이너 연산이 null 근처 주소 쓰기 위반
원인	check() 안에 넣은 상태 변경 호출이 NDEBUG에서 표현식짜 컴파일 제거 → 빈 컨테이너에 후속 연산 실행. 크래시 주소를 슬롯 오프셋 계산과 대조해 수치로 확정
해결	상태 변경을 독립 문장으로 분리하는 규약화 + 검증 파이프라인에 Release 등가(NDEBUG) 구성 상시 포함

6 검증 방법론

- 86개 단위 테스트 — Phase마다 선작성, 회귀 검용. 컨테이너 처닝(100키×10라운드), 순환 참조, 4~8 스레드×수십만회 참조 카운트 스트레스 포함
- **Windows Debug + Release** 필수 통과 — 구성 의존적 버그(위 트러블슈팅 ①③)를 놓치지 않기 위한 상시 검증
- **Linux g++ 3구성(O0 / O2 / O2+NDEBUG) + valgrind memcheck** — 다른 컴파일러·다른 힙 구현으로 이식성과 메모리 안전성 이중 확인

7 로드맵

- ✓ Phase 0-7.7 — Core: Memory / Templates / Containers / Math / TMap·TSet / String / Logging / SmartPointer / UObject / Timer / Gameplay Ability System
 - ✓ Phase 7.5+ — Core 최적화 5종: FMallocBinned · TInlineAllocator · TSparseArray+해시체인 · FNamePool · TSharedPtr 원자화
 - ▶ **Phase 8 — Renderer (DirectX11): DXDevice / SwapChain / SpriteBatch / RenderQueue / FCamera2D, WZ Canvas→텍스처 병행**
 - Phase 9-15 — Animation · Physics(Foothold) · Audio · UI · Input · Resource · World
 - LAYER 3 — 캐릭터 · 스킬(GAS 활용) · 몬스터 AI · 인벤토리 · 퀘스트 · Save/Load
-

MapleStory DX11 2D Engine · STL/예외/RTTI 없이 C++17로 직접 구현한 언리얼 아키텍처 재현 엔진 · 상세 구현 원리는 ARCHITECTURE.md, 트러블슈팅 전체 기록은 PROJECT_SUMMARY.md 참고